

تمرین سوم

برنامه نویسی پیشرفته

دکتر ملکی مجد

۱۷ اسفند ۱۴۰۳

زمستان - بهار ۴۰۳۲



دانشگاه علم و صنعت ایران

نکات مهم تمرین

- تنها زبان برنامه‌نویسی مجاز برای این تمرین سی شارپ است.
- از پاسخ تحویل داده شده تمرین ها ارائه گرفته خواهد شد.
- در صورت مشاهده هرگونه کپی برداری هم کپی کننده و هم کپی شونده نمره صفر خواهند گرفت.
- در صورت مشاهده استفاده از هرگونه LLM برای پاسخ گویی به سوالات به جای شما، نمره شما صفر خواهد شد.
- برای هر سری تمرین سه روز مهلت تحویل با تاخیر دارید و مجموعاً ده روز مهلت ارسال با تاخیر دارید.

طراحان تمرین

در صورت بروز هرگونه مشکل یا سوالی در مورد تمرین، می‌توانید با طراحان تمرین تماس بگیرید.

- سوال اول و دوم - هلیا شمس زاده @helia_shmsz
- سوال سوم و چهارم - سید علی حسینی @sseeyeedd_ir
- کسرا خلج - سوال پنجم @Uvaxztra

نکته مهم تمرین

در این تمرین به بعد بایستی مدیریت خطا در برنامه‌های خود را به خوبی انجام دهید. برای این کار باید از استثناءها استفاده کنید و همچنین باید پیغام‌های مناسبی برای کاربران برنامه‌تان نمایش دهید. در صورت عدم رعایت این نکته، نمره شما کاهش خواهد یافت.

۱ کار با فایل مقدماتی

در این تمرین قصد داریم اطلاعات یک فایل متنی را با کمی تغییرات به یک فایل جدید منتقل کند. به این صورت که در فایل جدید تمامی فضاها ی خالی بین کلمات تبدیل به * شده‌اند. همچنین باید در خروجی کنسول:

۱. تعداد خطوط

۲. تعداد اعداد

۳. تعداد ستاره‌ها

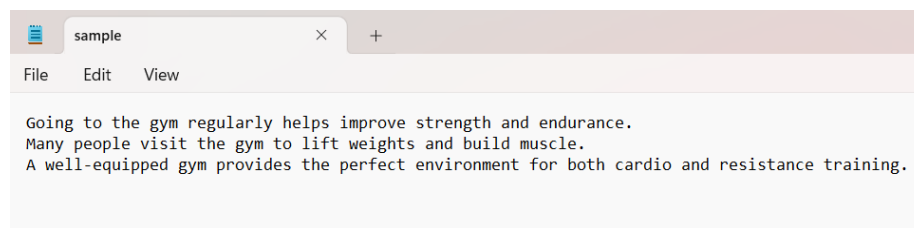
۴. تعداد حروف صدا دار فایل (a, e, i, o, u)

۵. تعداد کلماتی که با p شروع و به e ختم می‌شوند

۶. تعداد کلمه "gym" در متن (بزرگی و کوچکی حروف را در نظر نگیرید به این معنا که اگر در متن هم کلمه "gym"، هم "Gym" و هم "gYm" داشته باشیم، خروجی باید عدد ۳ باشد)

نمایش داده شود.

۱.۰.۱ مثال ۱:

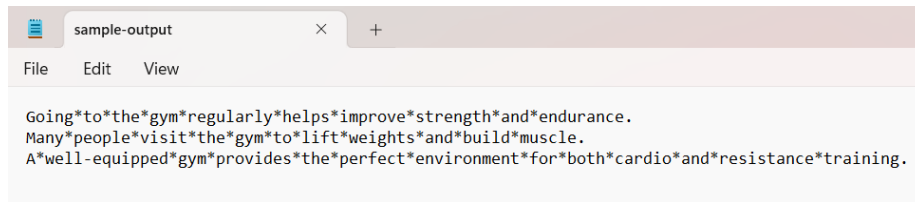


شکل ۱: فایل ورودی

فایل خروجی:

خروجی کنسول:

```
Number of lines : 3
Number of stars : 38
Number of digits : 0
Number of vowel sounds : 61
```



شکل ۲: فایل خروجی

Number of words start with 'p' and end with 'e' : 1
Number of "gym" : 3

۲ سوال ۲ - مدیریت باغ وحش

مسئول يك باغ وحش می‌خواهد برای نگهداری حیوانات از برنامه مدیریت اطلاعات استفاده کند و به کمک شما نیاز دارد. به همین منظور نیاز است کلاسی برای مدل کردن اطلاعات حیوانات بنویسید. هر حیوان یک شی از این کلاس می‌باشد. نوع حیوانات موجود در باغ وحش به شرح زیر است:

- میمون (Monkey)
- شیر (Lion)
- فیل (Elephant)
- خرس (Bear)
- ببر (Tiger)
- زرافه (Giraffe)

شما باید کلاسی به نام Zoo برای نگهداری اطلاعات حیوانات این باغ وحش طراحی کنید. اطلاعات هر حیوان شامل موارد زیر است:

۱. کد حیوان (ID)
۲. نوع حیوان (type)
۳. نام مستعار حیوان (name)
۴. محل نگهداری (location)
۵. نوع غذاهایی که می‌خورد (food)

همچنین یک فیلد مختص به کلاس وجود دارد که تعداد حیوانات موجود در باغ وحش را نشان می‌دهد، عنوان این فیلد (number) است. این اطلاعات فیلدهای کلاس Zoo را تشکیل می‌دهند.

توجه ۱: ویژگی کد هر حیوان و همچنین تعداد حیوانات موجود در باغ وحش نباید خارج از کلاس در دسترس باشند.

توجه ۲: نوع هر حیوان باید مقداری از یک نوع داده از enum باشند که خودتان تعریف کرده اید.

نکته ۱: نام هیچ دو حیوانی یکسان نیست. به عبارتی در هنگام ساخت هر حیوان این مورد باید چک شده و در صورتی که چنین نباشد، خطایی با پیغام مناسب نمایش داده شود. این عمل با استفاده از IsValidName صورت می‌گیرد که در ادامه توضیح داده می‌شود.

نکته ۲: کد هر حیوان همان تعداد حیوانات موجود در باغ وحش پس از اضافه شدن این حیوان است. به عنوان مثال، اگر ۱۰ حیوان در باغ وحش وجود داشته باشند، پس از اضافه شدن حیوان جدید، این حیوان کد ۱۱ را دریافت خواهد کرد.

نکته ۳: در هیچ جایی از کد برنامه‌تان اسم حیوانات را به صورت رشته نگهداری نکنید و فقط از enum ای که تعریف کرده اید استفاده نمایید. حتی در هنگام دادن ورودی به عنوان پارامتر های ورودی سازنده تابع این مورد را لحاظ کنید. در غیر این صورت نمره این بخش را از دست می‌دهید. مقادیر عددی هریک از موارد موجود در enum به شرح زیر است:

(میمون = ۱۰) - (شیر = ۱۱) - (فیل = ۱۲) - (خرس = ۱۳) - (ببر = ۱۳) - (زرافه = ۱۵)
توجه ۳: در صورت نیاز می توانید فیلد های دیگر را نیز به کلاس اضافه کنید.

۱.۲ کلاس Zoo:

پارامتر های سازنده کلاس، عبارت اند از: نوع حیوان (type)، نام حیوان (name)، محل نگهداری (location) و همچنین نوع غذایی که می خورد (food) را بگیرد.

۱.۱.۲ متد های موجود در کلاس:

۱. **متد SaveToFile:** اطلاعات حیوانات را در فایل ذخیره می کند. برای انجام این کار بایستی به ازای هر نوع حیوان یک فایل جداگانه بسازید. به عبارتی فایل با عنوان ۱۱ مربوط به حیواناتی از نوع شیر می باشد. در هر یک از این فایل ها، اطلاعات حیوان شامل آیدی، نام مستعار حیوان، محل نگهداری حیوان و نوع غذاهایی که می خورد ذخیره سازی می شوند. همچنین هر حیوان را با قرار دادن یک خط فاصله از حیوان بعدی جدا کنید. در صورتی که تعداد غذاهایی که یک حیوان می خورد از یک بیشتر باشد، هر مورد را بایک خط تیره از یکدیگر جدا کنید.

۲. **متد IsValidName:** این متد جهت چک کردن معتبر بودن نام مستعار هر یک از حیوانات است. پارامتر ورودی این متد نام حیوان است. نام معتبر نامی است که تنها شامل حروف الفبای انگلیسی به صورت بزرگ یا کوچک باشد و همچنین تاکنون تکرار نشده باشد.

۳. **متد Change:** سه متد با این نام ولی با پارامتر های ورودی مختلف وجود خواهد داشت. در هر یک از آنها ورودی اولی نام حیوان خواهد بود. سایر پارامتر های متد هارا با توجه به نیاز آنها تعیین کنید.

- متد اول: جهت تغییر محل نگهداری حیوان است.
- متد دوم: جهت تغییر نوع غذاهایی که می خورد است.
- متد سوم: جهت تغییر محل نگهداری به همراه نوع غذاهایی که می خورد.
- **نکته ۴:** تغییرات ایجاد شده در این متد روی فایل مربوط به حیوان نیز اعمال شود.
- ۴. **متد AllInfo:** که به صورت استاتیک تعریف می شود و به عنوان خروجی یک فایل با همین نام تولید می شود که در آن اطلاعاتی شامل:
 - تعداد حیوانات برای هر نوع حیوان از حیوانات
 - مجموع تعداد حیوانات موجود در باغ وحش
 - تعداد حروف صدا دار موجود در نام های حیوانات موجود برای هر نوع حیوان

۲.۲ در قسمت main این سوال:

دقت کنید که باید پیغام مناسب برای راهنمایی کاربر هنگام دریافت ورودی و نمایش خروجی چاپ کنید. ترتیب کارهایی که انجام می دهد به این صورت است:

- در ابتدا تعداد حیواناتی که قرار است به باغ وحش اضافه کنید را از ورودی دریافت کنید. سپس در خطوط بعدی اطلاعات حیوانات از ورودی دریافت کنید.
 - پس از دریافت اطلاعات هر حیوان، شی متناظر از این حیوان ساخته می شود. پس از آن با صدا زدن تابع SaveToFile این حیوان را داخل فایل مربوطه ذخیره سازی کنید.
 - در انتها باید متد AllInfo را که صدا بزنید.
- **نکته ۶:** هر غذا را هم در هنگام اضافه کردن حیوان و هم در هنگام تغییر اطلاعات مربوط به آن ها با استفاده از یک comma (,) از یکدیگر جدا کنید.

۱.۲.۲ نمونه‌ی ورودی و خروجی ها به صورت زیر است:

```
Enter the number of animals to save:
1
Enter the type:
Lion
Enter the Name:
Alex
Enter the location:
Block1
Enter the food:
Sushi,Steak
It's time to change information.
Enter the number of animals you want to change their information:
2
Wrong! Total number of animals in the zoo is 1. Enter a smaller
number:
1
Enter the name:
Alex
Enter the new food: (if you do not want to change it press enter)
Enter the new location: (if you do not want to change it press enter)
None of the values has not entered. Enter the values again:
Enter the new food: (if you do not want to change it press enter)
Enter the new location: (if you do not want to change it press enter)
Block3
```

شکل ۳: ورودی و خروجی

۳ ارباب فایل ها

۱.۳ مقدمه

در پردازش متن، جستجو، جایگزینی و ویرایش داده‌ها از عملیات‌های رایج محسوب می‌شوند. در این تمرین، برنامه‌ای پیاده‌سازی خواهید کرد که قادر به پردازش فایل‌های متنی در یک پوشه‌ی مشخص است. کلاس‌ها

۲.۳ FolderProcessor کلاس

این کلاس نماینده‌ی یک پوشه (دایرکتوری) بوده و لیستی از فایل‌های متنی (txt) را در خود نگه می‌دارد. تمام عملیات‌های پردازشی روی مجموعه‌ی فایل‌ها از این کلاس انجام می‌شود.

۱.۲.۳ ویژگی‌ها

- Files: یک آرایه ای از TextFileProcessor که تمام فایل‌های متنی را در خود نگه می‌دارد. ص
- Path: مسیر پوشه‌ی ورودی

۲.۲.۳ متدها

- **کانستراکتور** FolderProcessor(string path) با گرفتن مسیر پوشه، تمام فایل‌های متنی را در آن پوشه پیدا کرده و آرایه آن‌ها را در فیلد Files ذخیره می‌کند.
- FindFilesContainingText(string text) تمام فایل‌هایی که شامل متن مشخصی هستند را پیدا کرده و لیست آن‌ها را برمی‌گرداند.
- ReplaceTextInAllFiles(string oldText, string newText) یک متن را در تمام فایل‌های موجود جایگزین می‌کند.
- RemoveTextFromAllFiles(string text) متن موردنظر را از تمام فایل‌ها حذف می‌کند.
- CountOccurrencesInAllFiles(string text) تعداد دفعات تکرار یک متن را در کل فایل‌های پوشه شمارش می‌کند.
- GetFilesLength() طول محتوای متنی تمام فایل‌ها را برمی‌گرداند.
- ConvertAllToUppercase() تمام متن‌های داخل فایل‌ها را به حروف بزرگ تبدیل می‌کند.
- RemoveEmptyLinesFromAllFiles() تمام خطوط خالی را از فایل‌ها حذف می‌کند.
- AppendTextToAllFiles(string text) یک متن را به انتهای همه‌ی فایل‌ها اضافه می‌کند.

۳.۳ کلاس `TextFileProcessor`

این کلاس نماینده‌ی هر فایل متنی است و تمامی عملیات‌های ویرایش، جستجو و پردازش را روی خود انجام می‌دهد.

۱.۳.۳ ویژگی‌ها

- `Path`: مسیر فایل
- `Content`: محتوای متنی فایل
- `Size`: اندازه‌ی فایل بر حسب بایت

۲.۳.۳ متدها

- **کانستراکتور** `TextFileProcessor(string path)` این کانستراکتور یک پارامتر می‌گیرد که آدرس فایل هست و در این کانستراکتور فیلد‌های مربوطه پر میشوند.
- `ContainsText(string text)` بررسی می‌کند که آیا متن موردنظر در فایل وجود دارد یا خیر.
- `ReplaceText(string oldText, string newText)` تمام نمونه‌های `oldText` را با `newText` جایگزین می‌کند.
- `RemoveText(string text)` متن موردنظر را از محتوای فایل حذف می‌کند.
- `CountOccurrences(string text)` تعداد دفعات تکرار متن را در این فایل برمی‌گرداند.
- `GetLength()` سایز فایل را بر حسب بایت برمی‌گرداند.
- `ConvertToUppercase()` محتوای فایل را به حروف بزرگ تبدیل می‌کند.
- `RemoveEmptyLines()` تمام خطوط خالی را از متن فایل حذف می‌کند.
- `AppendText(string text)` یک متن را به انتهای فایل اضافه می‌کند.

۴.۳ روند اجرای برنامه

۱.۴.۳ دریافت مسیر پوشه‌ی ورودی

- برنامه از کاربر مسیر یک پوشه را دریافت می‌کند.

- تمام فایل‌های txt. در این مسیر شناسایی و در لیست Files ذخیره می‌شوند.

۲.۴.۳ اجرای عملیات پردازشی

- کاربر می‌تواند یکی از دستورات زیر را اجرا کند:
- جستجوی فایل‌هایی که شامل یک متن خاص هستند:
Find "example"
- جایگزینی متن در تمام فایل‌ها:
Replace "old" "new"
- حذف یک متن از تمام فایل‌ها:
Remove "target"
- شمارش تکرار یک متن در کل فایل‌ها:
Count "word"
- نمایش طول محتوای هر فایل:
Length
- تبدیل تمام متن‌ها به حروف بزرگ:
Uppercase
- حذف خطوط خالی از تمام فایل‌ها:
RemoveEmptyLines
- افزودن یک متن به انتهای تمام فایل‌ها:
Append "New Footer"

۵.۳ نکات مهم

- برنامه باید مدیریت خطای کامل داشته باشد تا در صورت عدم وجود فایل یا بروز مشکل، کرش نکند.
- برای خطاهای مختلف باید پیام‌های متناسب نمایش داده شود.
- نمایش نتیجه‌ها به کاربر باید به طور کامل انجام شود.

۴ مینیفایر ساده‌ی HTML، CSS و JS

۱.۴ مقدمه

در توسعه‌ی وب، **فشرده‌سازی (Minification)** یکی از تکنیک‌های مهم برای بهینه‌سازی سرعت بارگذاری صفحات است. این فرآیند شامل حذف فاصله‌های اضافی، کامنت‌ها و کاراکترهای غیرضروری از کدهای HTML، CSS و JavaScript می‌شود و باعث کاهش حجم این فایل‌ها خواهد شد.

در این تمرین، شما باید یک مینیفایر ساده برای فایل‌های HTML، CSS و JS بسازید. برنامه یک فایل را دریافت کرده و نسخه‌ی فشرده‌شده‌ی آن را در همان مسیر، اما با پسوند **.min** ذخیره می‌کند. علاوه بر این، تاریخچه‌ی تمام مینیفای‌های انجام‌شده باید در فایل **history.txt** ذخیره شود و امکان بازخوانی و نمایش آن نیز فراهم گردد.

۲.۴ Enum نوع فایل

برنامه باید نوع فایل را تشخیص دهد و بر اساس آن پردازش را انجام دهد:

- HTML
- CSS
- JS
- **Unsupported** (در صورت پشتیبانی نشدن فرمت)

۳.۴ کلاس TargetFile

۱.۳.۴ ویژگی‌ها

این کلاس شامل فیلدهای زیر خواهد بود:

- مسیر فایل اصلی
- حجم فایل قبل از فشرده‌سازی
- حجم فایل بعد از فشرده‌سازی
- تاریخ و زمان فشرده‌سازی
- نوع فایل (enum)

۲.۳.۴ متدها

۱.۲.۳.۴ Minify() این تابع وظیفه‌ی فشرده‌سازی فایل را به عهده دارد و بسته به نوع فایل عملیات مختلفی انجام می‌دهد:

اگر فرمت فایل **html** باشد:

- حذف کامنت‌ها:

- کامنت‌های چندخطی:

```
<!-- comment -->
```

- تبدیل فاصله‌های چندتایی به یک فاصله

- حذف کاراکترهای \n (خط جدید) و \t (تب)

- اگر فرمت فایل css باشد:

- حذف کامنت‌ها:

- کامنت‌های چندخطی:

```
/* Comment */
```

- تبدیل فاصله‌های چندتایی به یک فاصله

- حذف خطوط جدید (\n) و تب‌ها (\t)

- اگر فرمت فایل js باشد:

- حذف کامنت‌ها:

- کامنت‌های تک خطی:

```
// comment
```

- کامنت‌های چندخطی:

```
/* comment */
```

- تبدیل فاصله‌های چندتایی به یک فاصله

- حذف خطوط جدید (\n) و تب‌ها (\t)

- اگر نوع فایل پشتیبانی نشود، پیام مناسب نمایش داده شود.

پس از موفقیت‌آمیز بودن Minify()، خروجی در فایلی جدید با نامی مشابه ذخیره می‌شود:

style.css → style.min.css

script.js → script.min.js

index.html → index.min.html

و سپس تابع SaveToHistory() برای ذخیره‌سازی در تاریخچه فراخوانی می‌شود.

SaveToHistory() ۲.۲.۳.۴

- این تابع به صورت private پیاده سازی شده و اطلاعات فایل را در history.txt ذخیره می کند.
- مثال محتوای تاریخچه:

File: styles.css | Type: CSS | Original: 12 KB | Minified: 8 KB | Date: 2025-02-22 14:35

GetHistory() ۳.۲.۳.۴

- این تابع به صورت static پیاده سازی شده و محتوای history.txt را خوانده و نمایش می دهد.
-

۴.۴ روند اجرای برنامه

۱.۴.۴ دریافت مسیر فایل برای فشرده سازی

- کاربر مسیر یک فایل را وارد می کند.
 - برنامه نوع فایل را تشخیص می دهد (HTML، CSS، JS).
 - در صورت پشتیبانی، تابع Minify() اجرا می شود.
-

۲.۴.۴ نمایش تاریخچه ی فشرده سازی ها

اگر کاربر دستور history را وارد کند، تابع GetHistory() اجرا شده و لیست فایل های فشرده شده نمایش داده می شود.

۵.۴ نکات مهم

- برای خوانایی بهتر، تابع Minify() را با تقسیم وظایف به چند تابع private مجزا ساختار بندی کنید تا کد خوانا تر و توسعه پذیر تر باشد.
- برای بررسی ساینز فایل می توانید از System.IO یا طول متن فایل استفاده کنید.
- مدیریت خطاها الزامی است؛ برنامه نباید در هیچ شرایطی کرش کند و باید پیام های مناسب نمایش داده شود.
- فایل های HTML، CSS و JS جهت تست در اختیار شما قرار خواهند گرفت.

۵ سیستم پیک فضایی (Galactic Space Delivery)

۱.۵ توضیحات کلی:

شما یک سیستم پیک‌رسانی فضایی را پیاده‌سازی خواهید کرد. سفینه‌های فضایی را دارا هستید و کنترل می‌کنید و باید آنها را بین سیارات و یک ایستگاه مرکزی حرکت دهید، بارها را جمع‌آوری و تحویل دهید و سوخت را مدیریت کنید. ممکن است سفینه‌های شما با رویدادهای تصادفی مانند نشت سوخت یا حمله دزدان فضایی هنگام وارد شدن به یک سیاره مواجه شوند. هدف این است که شما با تحویل بارها یا کارگوها، پاداش جمع‌آوری کنید و با چالش‌های پیش‌بینی نشده مواجه شوید.

۲.۵ کلاس‌ها:

کلاس‌های زیر باید داخل برنامه پیاده‌سازی شوند: نکته: اسم‌ها کاملاً اختیاری‌اند. همین‌طور می‌توانید فیلدهای float را int در نظر بگیرید.

۱.۲.۵ سیاره (Planet):

دارای شناسه (int PlanetId)، نام، مختصات (int X, int Y, int Z)، ایستگاه سوخت فضایی (bool GalacticFuelStation)، و لیست بارهای موجود (List<Cargo> AvailableCargos). (منظور از بارهای موجود بارهایی است که در آن سیاره قرار دارد و به هیچ سفینه‌ای اساین نشده است. اگر باری به سفینه‌ای اساین یا داده شود، باید از این لیست حذف شود. توجه کنید که باید سفینه مورد نظر برای دریافت کارگو داخل سیاره مربوطه حضور داشته باشد)

۱.۱.۲.۵ متدها:

- **کانستراکتور:** این کلاس شامل یک کانستراکتور است که با گرفتن فیلدهای مربوطه، سیاره را می‌سازد.
- **PlanetStatus:** در ورودی این متد شناسه یک سیاره داده خواهد شد. باید تمام اطلاعات این سیاره نمایش داده شود.
- **DestroyPlanet:** در ورودی این متد شناسه یک کارگو داده خواهد شد، آن را از صحنه روزگار حذف می‌کند! اما اگر کارگویی در این سیاره وجود دارد، یا این سیاره سیاره مقصد کارگوی خاصی است و یا سفینه‌ای در این سیاره حضور دارد، نباید سیاره منفجر شود و ارور مناسب نمایش داده شود.
- نکته: مطمئن شوید که فیلد شناسه سیاره حتماً یکتاست.

۲.۲.۵ ایستگاه مرکزی (Station):

دارای نام (string Name)، مختصات (int X, int Y, int Z)، لیست سفینه‌ها (List<Ship> Ships) و تمام پول‌های فعلی سفینه‌ها (TotalNetWorth). (در سیستم فقط یک ایستگاه مرکزی وجود دارد. مختصات آن را (0, 0, 0) در نظر بگیرید.)

۱.۲.۲.۵ متدها:

- **کانستراکتور:** این کلاس شامل یک کانستراکتور است که با گرفتن فیلدهای مربوطه، ایستگاه را می‌سازد.
- **AddShip:** امکان اضافه کردن سفینه جدید با استفاده از کانستراکتور سفینه وجود داشته باشد. تمام سفینه‌ها در ایستگاه مرکزی تولید می‌شوند.
- **DestroyShip:** امکان حذف یک سفینه وجود دارد اگر در ایستگاه مرکزی قرار داشته باشد و کارگو هم نداشته باشد.
- **TotalNetWorth:** تمام پول‌های فعلی سفینه‌ها باید با هم جمع شوند و این فیلد برورسانی شود. هر زمان که پول یک سفینه برورسانی شد این فیلد هم باید برورسانی شود.
- نکته مهم: اگر سفینه‌ای در ایستگاه مرکزی است، مکان فعلی آن null خواهد بود.

۳.۲.۵ بار (Cargo):

دارای شناسه (int CargoId)، نوع بار (enum CargoType)، وزن (float Weight)، حجم (float Volume)، سیاره مبدا (Planet PickupPlanet)، سیاره مقصد (Planet DestinationPlanet)، پاداش (Reward)، وضعیت تحویل (bool DeliveryStatus) و یکپارچگی (float Integrity). (به عنوان مثال اگر یکپارچگی یک کارگو درون یک سفینه ۸۰ درصد باشد، زمانی که به سیاره مقصد می‌رسد، ۸۰ درصد پاداش داده می‌شود).

۱.۳.۲.۵ متدها:

- **کانستراکتور:** این کلاس شامل یک کانستراکتور است که با گرفتن فیلدهای مربوطه، کارگو را می‌سازد و آن را به لیست کارگوهای سیاره مبدا اضافه می‌کند. وضعیت تحویل در کانستراکتور باید False باشد.
- نکته: مطمئن شوید که فیلد شناسه کارگو حتما یکتاست.
- **CargoStatus:** در ورودی این متد شناسه یک کارگو داده خواهد شد. باید تمام اطلاعات این کارگو نمایش داده شود.

۴.۲.۵ سفینه (Ship)

دارای شناسه (ShipId)، نام (string Name)، سوخت فعلی (float CurrentFuel)، حداکثر (float MaxFuel)، مکان فعلی (Planet CurrentPlanet)، پول فعلی (float NetWorth)، ظرفیت وزن (float MaxWeightCapacity)، حجم بار (float MaxVolumeCapacity) و لیست بارها (List<Cargo> Cargos).

۱.۴.۲.۵ متدها:

- **کانستراکتور:** این کلاس شامل یک کانستراکتور است که با گرفتن فیلدهای مربوطه، سفینه را می‌سازد.
- **ShipStatus:** در ورودی این متد شناسه یک سفینه داده خواهد شد. باید تمام اطلاعات این سفینه نمایش داده شود.

- **HeadTo:** در ورودی این متد شناسه یک سیاره داده خواهد شد. این متد سفینه را در صورت داشتن سوخت کافی، به سمت سیاره مقصد ورودی داده شده می‌رساند. برای بدست آوردن سوخت مورد نیاز برای حرکت، کوتاه‌ترین فاصله بین سیاره فعلی و سیاره مقصد را پیدا کنید و اگر سوخت فعلی بیش از آن است می‌توان به سمت آن سیاره حرکت کرد و سوخت فعلی به مقدار سوخت مورد نیاز کسر می‌شود و در غیر این صورت ارور مناسب نمایش داده شود.
- **Refuel:** در ورودی این متد شناسه یک سیاره داده خواهد شد. اگر در سیاره فعلی، ایستگاه سوخت فضایی وجود داشت، می‌توان با استفاده از پول فعلی سفینه، برای آن سوخت خرید. با هر ۵ واحد پول می‌توانیم ۱ واحد سوخت بخریم. به عنوان مثال اگر سفینه ۲۰۰ واحد پول دارد، می‌تواند ۴۰ واحد سوخت دریافت کند. دقت داشته باشید که نمی‌توان بیشتر از ماکسیموم گنجایش سوخت سفینه سوخت خرید یا سوخت اختیار کرد.
- **PickUpCargo:** در ورودی این متد شناسه یک کارگو داده خواهد شد. اول بایستی بررسی شود که آیا کارگوی داده شده داخل سیاره فعلی قرار دارد یا نه. سپس باید بررسی شود که آیا سفینه می‌تواند وزن و حجم کارگو را تحمل کند یا خیر. اگر وزن و حجم کافی وجود دارد باید وزن و حجم باقی‌مانده سفینه بروزسانی شود و کارگو به سفینه اساین شود.
- **DeliverCargo:** در ورودی این متد شناسه یک کارگو داده می‌شود. اول بایستی بررسی شود که مکان فعلی با سیاره مقصد کارگو مطابقت دارد یا نه، اگر نداشت باید ارور مناسب نمایش داده شود. سپس وضعیت تحویل کارگو True خواهد شد و کارگو از لیست کارگوهای سفینه خارج می‌شود و وزن و حجم سفینه بروزسانی خواهد شد و با توجه به یکپارچگی کارگو، پاداش داده خواهد شد و در نهایت پاداش نهایی به پول‌های فعلی سفینه اضافه خواهد شد.

۳.۵ رویدادهای تصادفی

- پس از هر حرکت سفینه با متد HeadTo، یک رویداد تصادفی با احتمال ۳۰٪ رخ می‌دهد. رویدادها شامل:
- **حمله دزدان فضایی:** ۱۰٪ سوخت کاهش می‌یابد و یکپارچگی همه بارها ۵۰٪ کم می‌شود.
 - **بارش شهابی:** یکپارچگی همه بارها ۱۵٪ کاهش می‌یابد.
 - **نشت سوخت:** ۲۰٪ از سوخت باقی‌مانده از دست می‌رود.
- این رویدادها باید در متد TriggerRandomEvent پیاده‌سازی شوند و پیام مناسبی به بازیکن نمایش دهند.

۴.۵ Main

- برنامه مین شما باید فیچرهای زیر را دارا باشد:
- مشاهده وضعیت‌ها (نمایش اطلاعات سفینه‌ها و بارها و سیاره‌ها).
 - اضافه کردن و حذف کردن کارگو.

- اضافه کردن و حذف کردن سفینه.
- اضافه کردن و حذف کردن سیاره.
- حرکت به مکان (سفینه به مکان مشخص برود).
- جمع‌آوری بار (از سیاره فعلی بار انتخاب کند).
- تحویل بار.
- سوختگیری.
- خروج.

۵.۵ جزئیات پیاده‌سازی:

از enum برای انواع بار (Food، Minerals، Technology، LuxuryGoods، Hazardous Weapons) استفاده کنید. شناسه‌ها (PlanetId، Cargold، ShipId) باید منحصر به فرد باشند. سیستم شامل چندین سیاره و ۱ ایستگاه مرکزی می‌باشد.

۶.۵ نکات مهم:

از Random برای شبیه‌سازی موارد غیرمنتظره و رویدادها استفاده کنید. باید تمام ارور های ورودی دهی و ورودی هی فانکشن ها و متد ها هندل شده باشند. همه ویژگی‌ها (از جمله رویدادهای تصادفی) باید پیاده‌سازی شوند. هرگونه موارد اضافه‌ای که در اجرای برنامه به شما کمک می‌کنند می‌توانند نوشته شوند

خروجی‌ها باید واضح باشند مثلاً `Pirate Attack! Lost 10 fuel`.